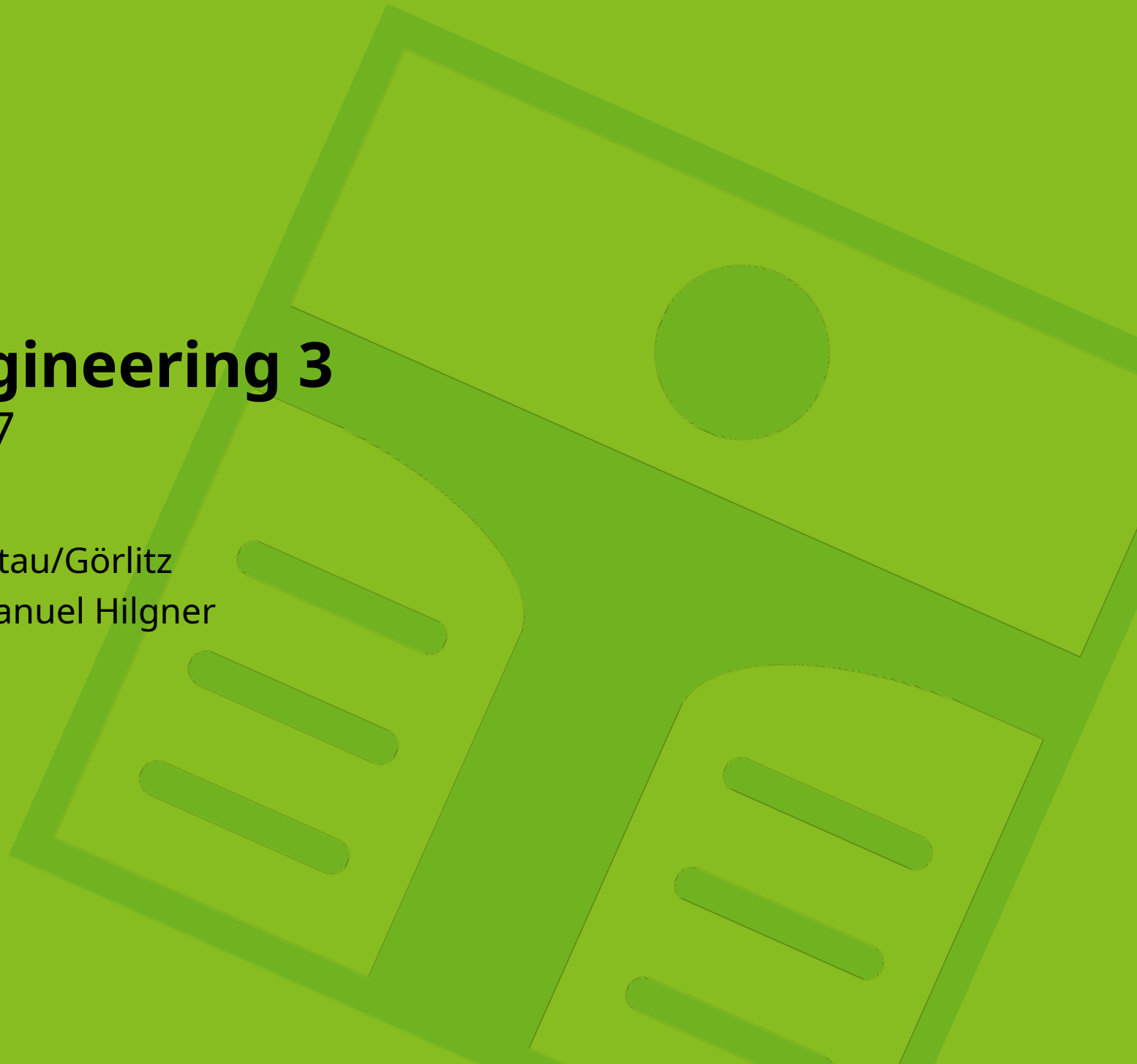


Web Engineering 3

Vorlesung 7

Hochschule Zittau/Görlitz

Christopher-Manuel Hilgner



Agenda

Vorlesung

- Error Handling in Spring

Seminar

- Error Handling
- Build Container für Spring Anwendung
- Container für Spring Anwendung

Custom Template Messages

Custom Template Messages

```
1 class ErrorMessageModel(  
2     var status: Int? = null,  
3     var message: String? = null  
4 )
```

 Kotlin

- status speichert den HTTP Status Code
- message speichert eine benutzerdefinierte Nachricht, die an das Frontend geschickt wird, wenn ein Error geworfen wird

Exception Controller Advice

Custom Template Messages

```
1  @ControllerAdvice Kotlin
2  class ExceptionControllerAdvice {
3      fun handleIllegalStateException(
4          ex: IllegalStateException
5      ): ResponseEntity<ErrorMessageModel> {
6          val errorMessage = ErrorMessageModel(
7              HttpStatus.NOT_FOUND.value(),
8              ex.message
9          )
10         return ResponseEntity(
11             errorMessage,
12             HttpStatus.BAD_REQUEST
13         )
14     }
15 }
```

Custom Template Messages

```
1 class ArticleNotFoundException(  
2     message: String  
3 ): RuntimeException(message) {  
4  
5 }
```

◀ Kotlin

- Es können eigene Exceptions erstellt werden
- Diese Exceptions sollten spezifische Szenarien im Backend abdecken

Custom Template Messages

```
fun handleArticleNotFoundException(ex:  
1 ArticleNotFoundException):  
  ResponseEntity<ErrorMessageModel> {  
2   val errorMessage = ErrorMessageModel(  
3     HttpStatus.NOT_FOUND.value(),  
4     ex.message  
5   )  
6   return ResponseEntity(  
7     errorMessage, HttpStatus.NOT_FOUND  
8   )  
9 }
```

[◀ Kotlin](#)

Einbindung

Einbindung

```
1 fun getArticle(id: String): ArticleModel {  
2     return articles.find { articleModel -> articleModel.id ==  
   id } ?: throw ArticleNotFoundException("Article not found")  
3 }
```

◀ Kotlin

Einbindung

```
1 fun createArticle(title: String): ArticleModel {  
2     val article = (articles.find { articleModel ->  
        articleModel.title == title })  
3     if (article != null) {  
4         throw IllegalStateException("Article with the same title  
            already exists")  
5     }  
6     return ArticleModel("4", title)  
7 }
```

 Kotlin

Einbindung

```
1 fun updateArticle(id: String, title: String):  
  ArticleModel {  
2     val article = (articles.find { articleModel ->  
3         articleModel.id == id  
4     } ?: throw ArticleNotFoundException("Article not found"))  
5     if (title.length > 50) {  
6         throw IllegalArgumentException("Article title too long")  
7     }  
8     article.title = title  
9     return article  
10 }
```

◀ Kotlin

ResponseStatusException Class

ResponseStatusException Class

```
1  @PostMapping
2  fun updateArticle(
3      @RequestParam id: String,
4      @RequestParam title: String
5  ): ArticleModel {
6      try {
7          return articleService.updateArticle(id, title)
8      } catch (ex: IllegalArgumentException) {
9          throw ResponseStatusException(
10             HttpStatus.BAD_REQUEST.localizedMessage, ex
11         )
12     }
13 }
```

 Kotlin

ResponseStatusException Class

- Gut um Exceptions dynamisch zu handeln
- Exceptions können in der Methoden Definition des Controllers verarbeitet werden
- Es wird kein globaler Controller benötigt
- Es müssen Orte definiert werden, wo spezifische Exceptions gehandelt werden

MENSA IST VORBEI